RACIOCÍNIO EM SISTEMAS DE LÓGICA CLÁSSICA –

PROPOSIÇÕES E INFERÊNCIAS

Prof. Wagner Igarashi, Dr.

Agentes

- O conhecimento dos ***agentes de resolução de problemas*** é muito específico e inflexível.
  - Exemplo: um agente de xadrez entende as regras do jogo, mas não tem consciência de restrições do mundo real, como a impossibilidade de um objeto ocupar dois lugares ao mesmo tempo.
- Os agentes baseados em conhecimento.
  - Podem combinar e recombinar informações para atender a uma infinidade de propósitos.
    - Exemplo: Eles podem combinar conhecimento geral com percepções correntes para deduzir aspectos ocultos do ambiente.
  - Conhecimento e raciocínio.
    - São cruciais ao lidar com ambientes parcialmente observáveis.

Agentes baseados em conhecimento

- Exemplo: Um médico faz o diagnóstico de um paciente antes do tratamento.
  - Doença não é diretamente observável.
  - Conhecimento utilizado
    - Uma parte em forma de regras aprendidas com livros e professores.
    - Uma parte em forma de padrões de associação no cérebro do médico.
- Exemplo : Reconhecimento de linguagem natural.
  - Exige dedução do estado oculto - a intenção do falante.
    - “John viu a joia pela janela e a desejou”.
    - “John lançou a pedra contra a janela e a quebrou”.
  - Raciocínio sobre um repertório de conhecimento de senso comum.
  - Agentes de resolução de problemas – representação de problemas de contingência inerentemente exponenciais

Agentes baseados em conhecimento

- Tem flexibilidade.
- São capazes de aceitar novas tarefas sob a forma de metas descritas de modo explícito.
- Podem alcançar competência rapidamente.
  - Ao serem informados ou,
  - Adquirirem novos conhecimentos sobre o ambiente.
- Podem se adaptar a mudanças no ambiente.
  - Atualizando o conhecimento relevante.

Representação de conhecimento

- A **LÓGICA** é o princípio da maioria da tecnologias de representação de conhecimento.
  - Exemplo simples de representação para agentes baseados em conhecimento.
  - Apresenta algumas **limitações severas**: não consegue representar muito bem nem incerteza nem tempo.
  - Lógica Proposicional (LP) e Lógica de Primeira Ordem (LPO)

Agentes baseados em conhecimento

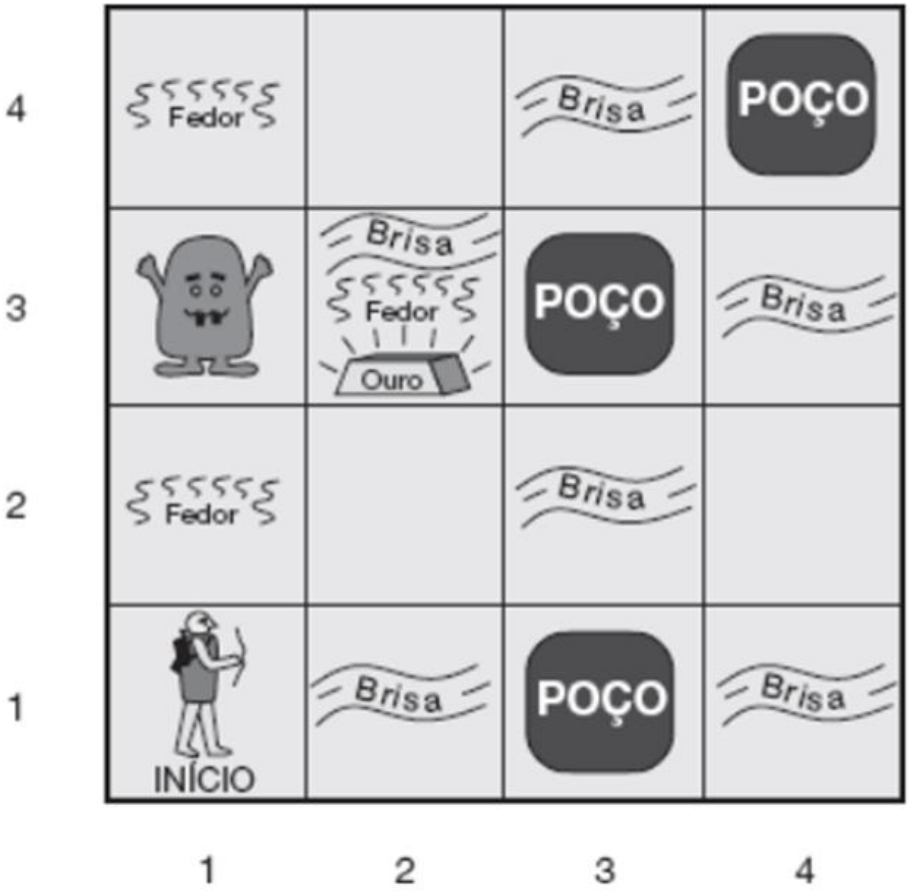
- Componente central – **BASE DE CONHECIMENTO** (BC)
  - É um conjunto de sentenças.
    - Expressas em uma linguagem de representação de conhecimento (LRC).
    - Representam alguma asserção (afirmação) sobre o mundo.
- Para adicionar sentenças à BC usa-se uma função chamada – ***TELL*** (informe).
- Para consultar o que se conhece usa-se uma função chamada – ***ASK*** (pergunte).
- *TELL* e *ASK* podem envolver **INFERÊNCIA**, i.e. derivação de novas sentenças a partir de sentença antigas pertencentes a BC.

Agentes baseados em conhecimento

- São semelhantes aos agentes com estado interno.
  - Mas não são um programa arbitrário para calcular ações.
  - Eles se adaptam a uma descrição no **nível de conhecimento**:
    - Em que precisamos especificar apenas o que o agente sabe e quais são suas metas, a fim de corrigir seu comportamento.
  - O nível de conhecimento é independente do **nível de implementação**:
    - Não estamos preocupados como o conhecimento é implementado: se é na forma de listas, mapas, *strings*, neurônios, etc.

O mundo do *Wumpus*

- Salas conectadas por passagens.
- *Wumpus* é um mostro devorador que pode ser morto pela <u>única</u> flecha do agente.
- Algumas salas contém poços sem fundos.
- O objetivo do jogo é encontrar o ouro.
- O agente morre se entrar em uma sala com um poço ou o *Wumpus* vivo.



Ambiente de tarefa – o Mundo do *Wumpus*

- Medida de desempenho
  - Pegar o ouro = +1000
  - Cair em poço ou ser devorado = -1000
  - Executar ação = -1
  - Usar a flecha = -10

- Ambiente
  - Um malha de salas 4x4.
  - Agente começa em [1,1], voltado para a direita.
  - Posições do ouro e do *Wumpus* são aleatórias.
  - Cada sala pode ter um poço com probabilidade de 0,2.
  - O quadrado inicial nunca tem *Wumpus*, ouro ou poço.

Ambiente de tarefa – o Mundo do *Wumpus*

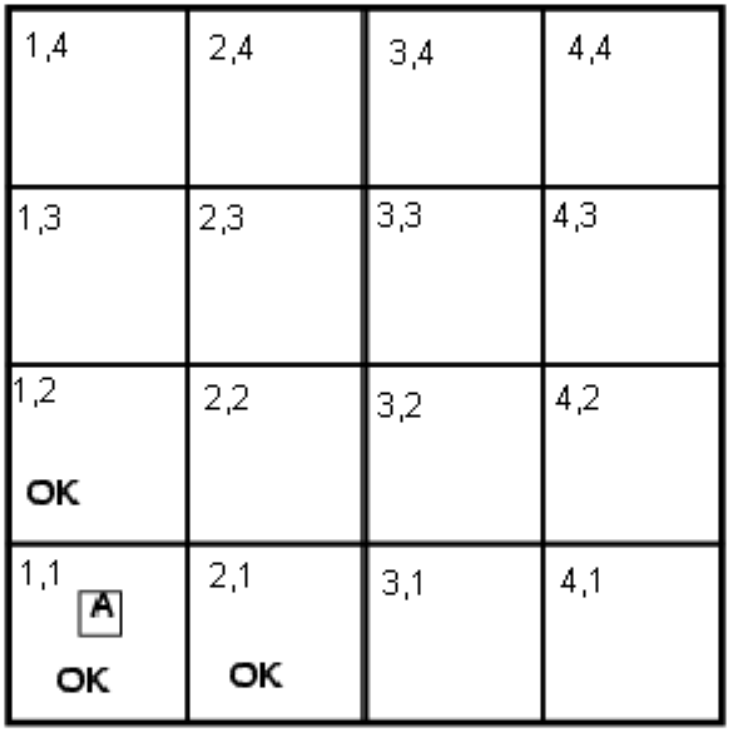
- Atuadores
  - Agente pode se mover para frente, ou a esquerda e a direita 90º.
  - Mover-se para frente não tem efeito se houver uma parede.
  - Ação ***agarrar*** – pega um objeto que está no mesmo quadrado que o agente.
  - Ação ***atirar*** – atira a flecha em linha reta diante do agente.
    - A flecha irá parar somente quando atingir o ***Wumpus*** ou a parede.
    - Como o agente só tem uma flecha apenas a primeira ação atirar tem algum efeito.

Ambiente de tarefa – o Mundo do *Wumpus*

- Sensores: fornecem um único bit de informação.
  - *Fedor* – quadrado com o *Wumpus* e adjacentes a ele.
  - *Brisa* – quadrados adjacentes a um poço.
  - *Brilho* – quadrado com o ouro.
  - *Impacto* – quando caminhar para uma parede.
  - *Grito* – do *Wumpus* quando morre.
    - Exemplo: [Fedor, Brisa, Nada , Nada , Nada]
- **Principal dificuldade** – ignorância inicial da configuração do ambiente – exige exploração e raciocínio.

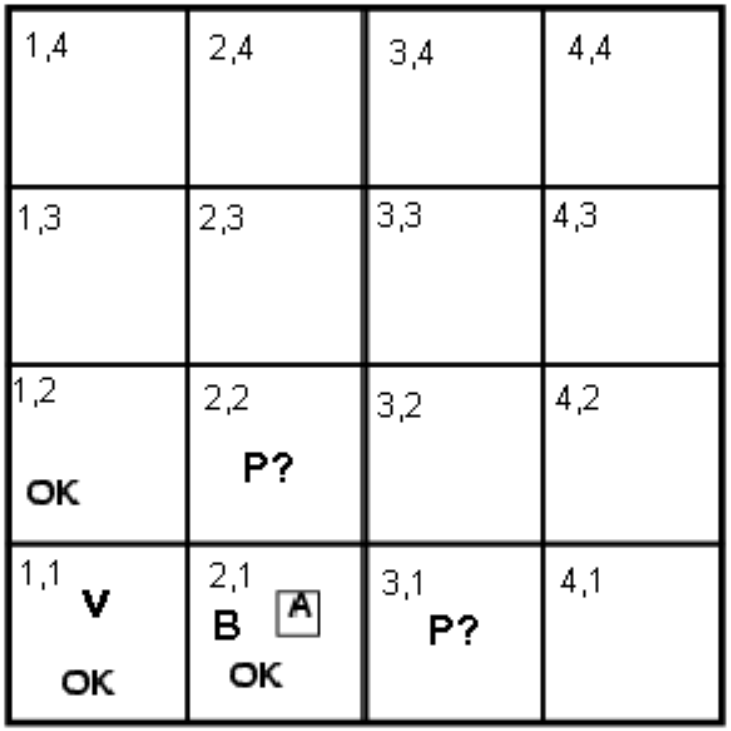
Explorando o mundo do *Wumpus*

- Inicialmente a BC contém somente as regras do ambiente.
- Em particular o agente sabe que está em [1,1] e este quadrado é seguro.
- A 1ª percepção:
  - [Nada, Nada, Nada, Nada, Nada]
- Então o agente pode concluir que os quadrados adjacentes são seguros.



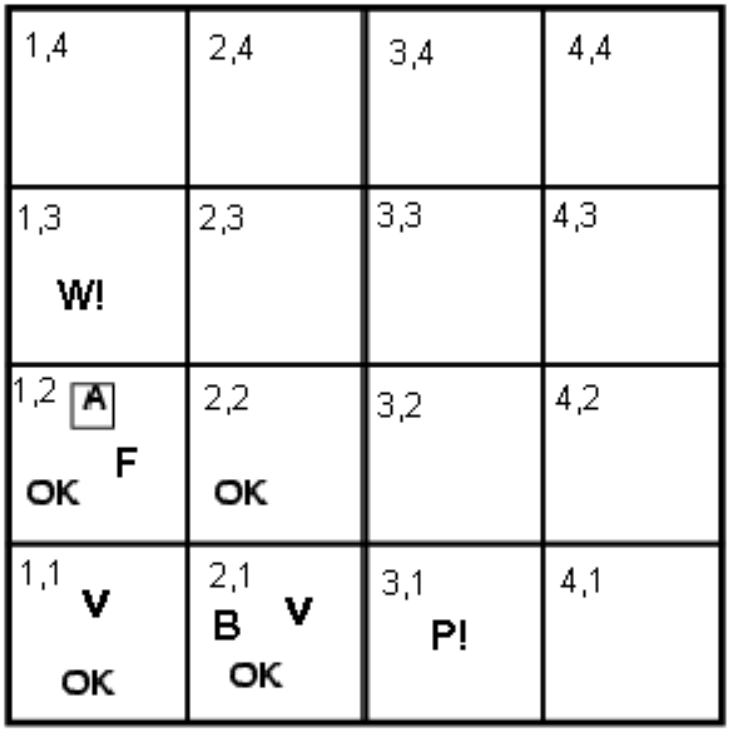
Explorando o mundo do Wumpus

- Como o agente já está virado para a direita, ele deve explorar [2,1].
- A percepção em [2,1] é:
  - [Nada, Brisa, Nada, Nada, Nada]
- Então pode existir um poço em [3,1] ou em [2,2].
- O agente não tem certeza se pode ir para [3,1] então ele deve voltar e explorar [1,2].



Explorando o mundo do Wumpus

- A percepção em [1,2] é:
  - [Fedor, Nada, Nada , Nada, Nada]
- Então pode existir um *Wumpus* em [1,3] ou em [2,2]
  - Se o *Wumpus* estivesse em [2,2], haveria Fedor em [2,1].
  - Então → *Wumpus* em [1,3].
- Se o poço estivesse em [2,2], haveria brisa em [1,2].
  - Então → poço em [3,1].



Explorando o mundo do Wumpus

- O único quadrado seguro é [2,2].
- A percepção em [2,2] é:
  - [Nada, Nada, Nada, Nada, Nada]
- Então [2,3] e [3,2] são seguros.
- O agente vai para [2,3] onde a percepção é:
  - [Fedor, Brisa, Brilho, Nada, Nada]
- Fedor em [2,3], confirma *Wumpus* em [1,3].
- Brisa em [2,3] indica que pode existir um poço em [2,4] ou [3,3].
- Brilho em [2,3], indica ouro em [2,3].
- O agente pega o ouro e encerra.

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A F B R	3,3 P?	4,3
1,2 F OK V	2,2 OK V	3,2	4,2
1,1 OK V	2,1 B OK V	3,1 P!	4,1

- **Como representar as informações do *Wumpus* e tirar conclusões sobre elas?**

Lógica

- **Sintaxe** da lógica → expressa as sentenças da BC de acordo com uma linguagem de representação de conhecimento.

- A sintaxe especifica as sentenças que são bem-formadas na linguagem:
  - Exemplificando, na matemática:
    - X + Y= 4 é uma sentença bem formada.
    - X2Y + = ***não*** é uma sentença bem-formada.

- O raciocínio envolverá a geração e manipulação destas sentenças.

Lógica

- **Semântica** da lógica → **significado** das sentenças.

- A semântica, na lógica, define a verdade de cada sentença em relação a cada **mundo possível**.
  - X + Y = 4
    - Verdadeira em um mundo em que X=2 e Y=2.
    - Falsa em um mundo em que X=1 e Y=1.

- Em lógicas clássicas uma sentença é Verdadeira ou Falsa em um mundo possível (modelo).
  - Não existe um meio termo.

Proposições

- Uma **INTERPRETAÇÃO** é uma atribuição de valores para todas as variáveis da BC.
- Um **MODELO** é uma interpretação que satisfaz as restrições de validade para a BC.
- Muitas vezes nós não queremos apenas encontrar um modelo, mas sim saber o que é verdade em todos os modelos.
- Uma **PROPOSIÇÃO** é uma afirmação que é *Verdadeira* ou *Falsa* em **cada interpretação possível**.

Exemplo simples

$$KB = \begin{cases} p \leftarrow q. \\ q. \\ r \leftarrow s. \end{cases}$$

	p	q	r	s	
I_1	*true*	*true*	*true*	*true*	is a model of KB
I_2	*false*	*false*	*false*	*false*	not a model of KB
I_3	*true*	*true*	*false*	*false*	is a model of KB
I_4	*true*	*true*	*true*	*false*	is a model of KB
I_5	*true*	*true*	*false*	*true*	not a model of KB

interpretações

$KB \models p$, $KB \models q$, $KB \not\models r$, $KB \not\models s$

Modelos e consequência lógica

- Um **MODELO** de um conjunto de cláusulas é uma interpretação na qual todas as cláusulas são verdadeiras.

- Se a BC é um conjunto de cláusulas e ***g*** é uma conjunção de átomos, ***g*** é uma **CONSEQUÊNCIA LÓGICA** da BC, escrito **BC |= g** se ***g*** é verdade em todos os modelos de BC.

- Ou seja, **BC |= g** se não houver nenhum interpretação no qual BC é verdadeira e ***g*** é falsa.

Consequência lógica e inferência

- Se pensarmos em todas as consequências lógicas de BC como um palheiro e **α** como uma agulha, o **ALGORITMO DE INFERÊNCIA** é o mecanismo para encontrar a agulha.

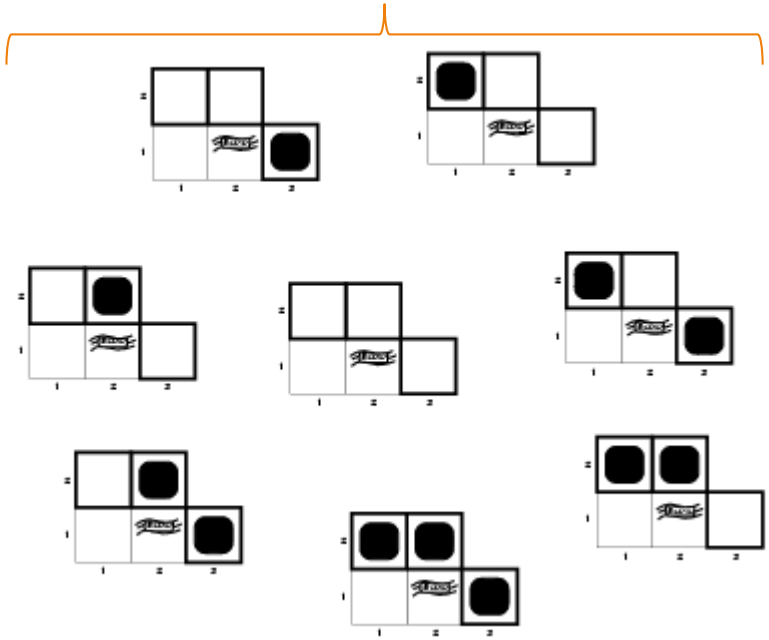
- A notação **BC $\vdash_i$ α** significa que o algoritmo de inferência ***i*** pode derivar **α** de BC.

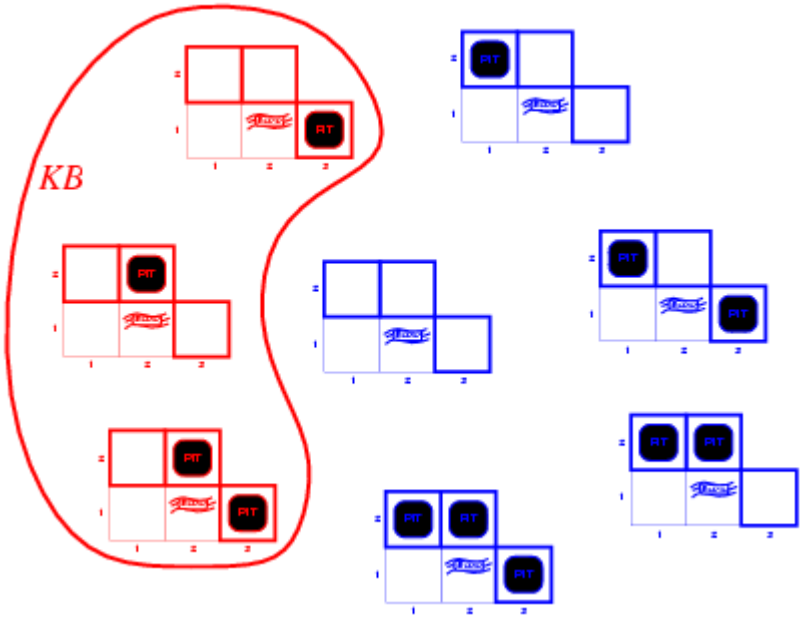
- Um algoritmo de inferência que deriva apenas sentenças válidas é **CONSISTENTE** ou **correto**.
  - BC $\vdash_i$ α implica que BC $\vDash$ α

- Um algoritmo de inferência é **COMPLETO** se puder derivar qualquer sentença válida.
  - BC $\vDash$ α implica que BC $\vdash_i$ α

Modelos para o mundo de Wumpus

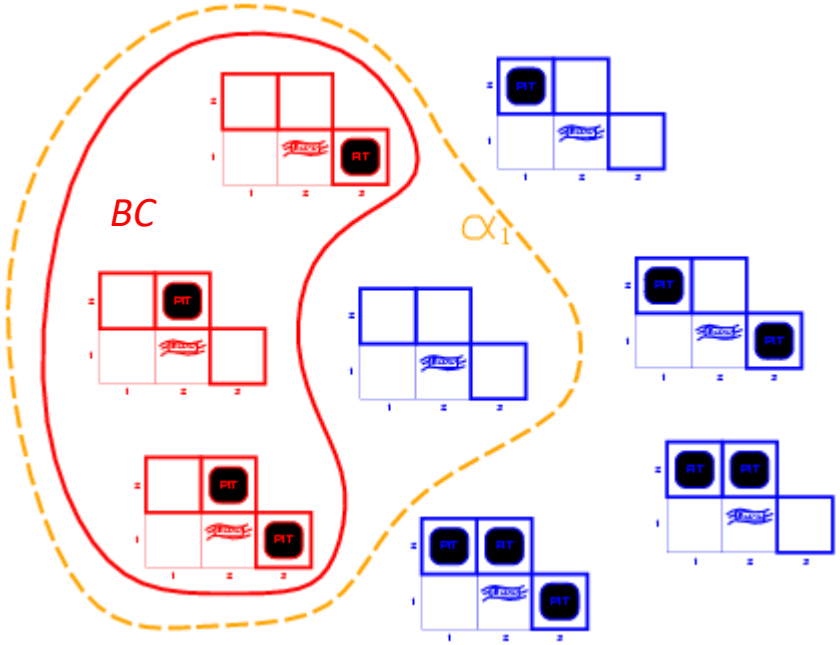
Mundos possíveis





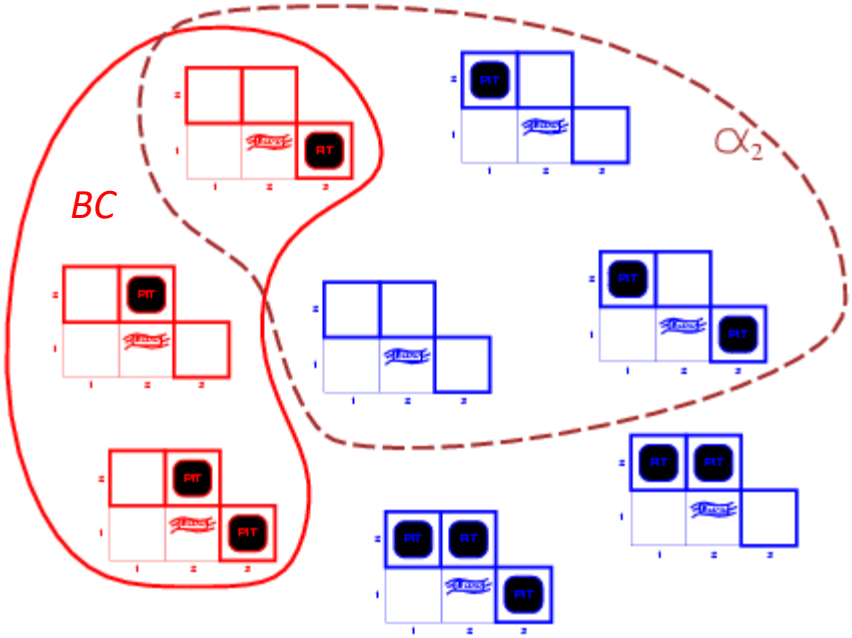
- KB ou BC = regras do mundo do *Wumpus* + observações

Modelos para o mundo de Wumpus



- **Dada a sentença**: α_1 = "não existe poço em [1,2]"
- **Como** α_1 é verdadeira em todos os modelos em que a base (KB) é verdadeira
- **Então** BC $\vDash \alpha_1$ (ou seja, BC tem como consequência lógica a sentença α_1)

Modelos para o mundo de Wumpus

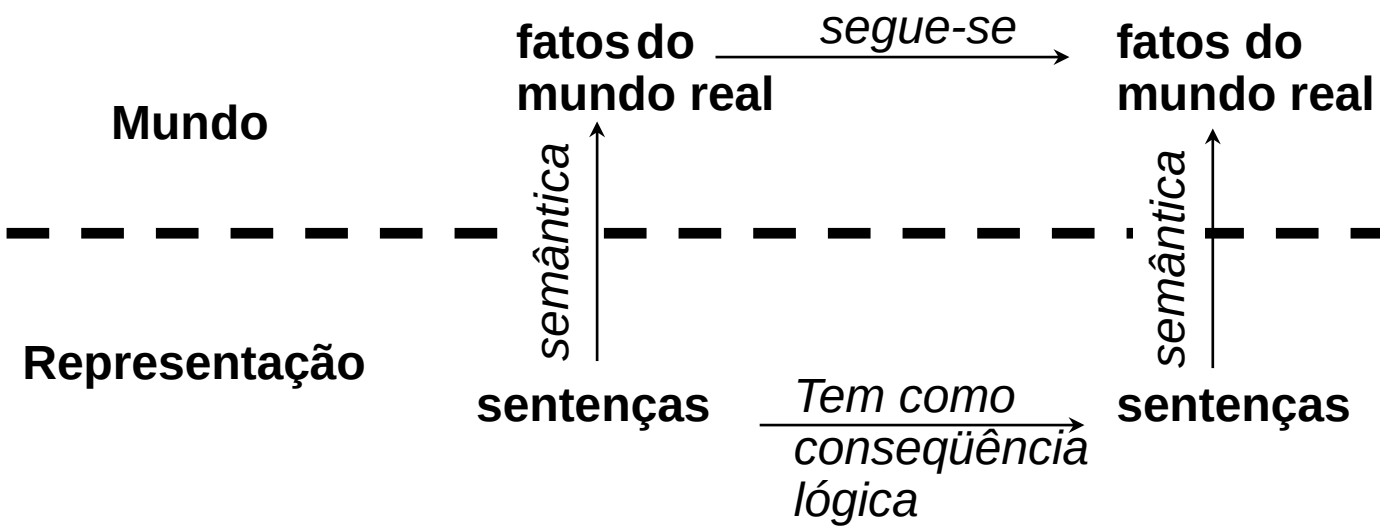


- **Dada a sentença**: α_2 = "não existe poço em [2,2]"
- **Como** α_2 não é verdadeira em todos os modelos em que a base (KB) é verdadeira
- **Então** BC $\nvDash \alpha_2$ (ou seja, KB não tem como consequência lógica a sentença α_2)

- O algoritmo de inferência apresentado nos 2 slides anteriores é chamado de **verificação de modelos**
  - pois enumera todos os modelos possíveis para **verificar** que
    - α é verdadeira em todos os modelos em que BC é verdadeira.

Lógica X Mundo real

- Se a BC é verdadeira no mundo real, qualquer sentença ***α*** derivada de BC por um procedimento de prova correto também será verdadeira no mundo real.



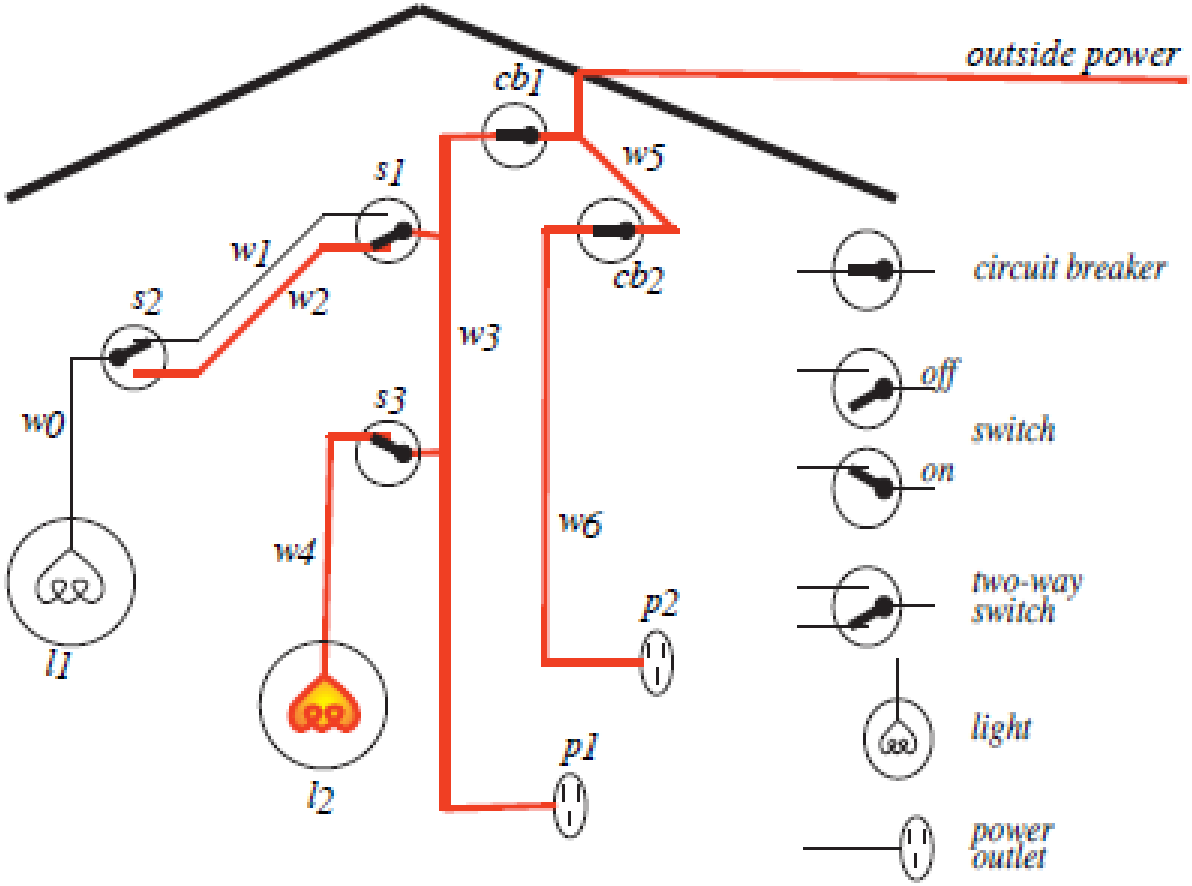
Visão do usuário da semântica

1. Escolha um domínio de tarefa: **interpretação pretendida**.
2. Associe um átomo a cada proposição que você pretende representar.
3. Informe (*tell*) ao sistema as cláusulas que são verdadeiras na interpretação pretendida: **axiomatização do domínio**.
4. Faça (*Ask*) perguntas sobre a interpretação pretendida.
5. Se **BC |= g**, então ***g*** deve ser verdadeira na interpretação pretendida.
6. Os usuários podem interpretar a resposta usando sua interpretação pretendida dos símbolos.

Visão do computador da semântica

- O computador não tem acesso à interpretação pretendida.
- Tudo o que ele sabe é a base de conhecimentos.
- O computador pode determinar se uma fórmula é uma consequência lógica da BC.
- Se **BC |= g** então ***g*** deve ser verdadeira na interpretação pretendida.
- Se **BC |≠ g**, então há um modelo na BC no qual ***g*** é falsa. Esta poderia ser a interpretação pretendida.

Exemplo: ambiente elétrico



O papel da semântica

No computador:

$light1_broken \leftarrow sw_up$

$\quad \wedge\ power \wedge unlit_light1.$

$sw_up.$

$power \leftarrow lit_light2.$

$unlit_light1.$

$lit_light2.$

Na mente do usuário:

- *light1_broken*: light #1 is broken
- *sw_up*: switch is up
- *power*: there is power in the building
- *unlit_light1*: light #1 isn't lit
- *lit_light2*: light #2 is lit

Conclusão: *light1_broken* (a luz1 está quebrada)

O computador não sabe o significado dos símbolos.
O usuário pode interpretar o símbolo usando seu significado.

Exemplo: representando o ambiente elétrico

$light_l_1.$

$light_l_2.$

$down_s_1.$

$up_s_2.$

$up_s_3.$

$ok_l_1.$

$ok_l_2.$

$ok_cb_1.$

$ok_cb_2.$

$live_outside.$

$lit_l_1 \leftarrow live_w_0 \wedge ok_l_1$

$live_w_0 \leftarrow live_w_1 \wedge up_s_2.$

$live_w_0 \leftarrow live_w_2 \wedge down_s_2.$

$live_w_1 \leftarrow live_w_3 \wedge up_s_1.$

$live_w_2 \leftarrow live_w_3 \wedge down_s_1.$

$lit_l_2 \leftarrow live_w_4 \wedge ok_l_2.$

$live_w_4 \leftarrow live_w_3 \wedge up_s_3.$

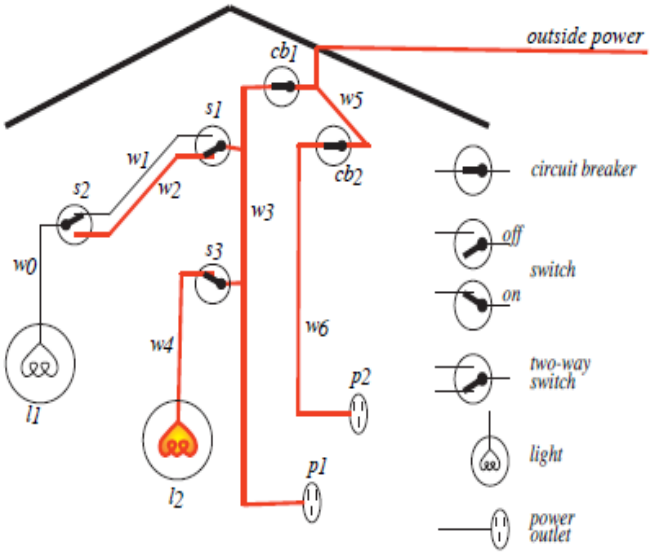
$live_p_1 \leftarrow live_w_3.$

$live_w_3 \leftarrow live_w_5 \wedge ok_cb_1.$

$live_p_2 \leftarrow live_w_6.$

$live_w_6 \leftarrow live_w_5 \wedge ok_cb_2.$

$live_w_5 \leftarrow live_outside.$



Lógica Proposicional ou Booleana

- É uma lógica muito simples.

- Sintaxe:
  - **SENTENÇAS ATÔMICAS** – elementos sintáticos indivisíveis – consistem em um **único** símbolo proposicional.
    - Representados com letras maiúsculas: P, Q, R, $W_{1,3}$, $P_{1,2}$ ...
  - Escolhemos nomes arbitrários para representar algum valor para o leitor.
    - $W_{1,3}$ para representar que o ***Wumpus*** está em [1,3].
    - $P_{1,2}$ para representar que existe um poço em [1,2].

Lógica Proposicional – **Sintaxe**

- Existem dois símbolos proposicionais com significados fixos:
  - Verdadeiro é a proposição sempre verdadeira, e
  - Falso é a proposição sempre falsa.
- **SENTENÇAS COMPLEXAS** são construídas a partir de sentenças simples + conectivos lógicos.
- **CONECTIVOS LÓGICOS**
  - ¬ (não) – **negação** de uma sentença. Ex.: $\neg W_{1,3}$
  - ∧ (e) – **conjunção** de duas sentenças. Ex.: $W_{1,3} \wedge P_{3,1}$
  - ∨ (ou) – **disjunção** de duas sentenças. Ex.: $P_{3,1} \vee P_{2,2}$

Lógica Proposicional – **Sintaxe**

- Conectivos lógicos
  - => (implicação) – **implicação** de duas sentenças.
    - Ex.: $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$
      - no qual $(W_{1,3} \wedge P_{3,1})$ é chamado premissa ou **ANTECEDENTE**.
      - e $\neg W_{2,2}$ é chamado conclusão ou **CONSEQUENTE**.
  - <=> se e somente se ou **bicondicional**.
    - Ex.: $W_{1,3} \Leftrightarrow \neg W_{2,2}$
  - Ordem de precedência (decrescente) dos conectivos é:
    - $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$
    - $\neg P \vee Q \wedge R \Rightarrow S$ → $((\neg P) \vee (Q \wedge R)) \Rightarrow S$

Lógica Proposicional – Semântica

- Define as regras para especificar a verdade de uma sentença em relação a um modelo específico.

- Um modelo fixa um valor verdade para um símbolo proposicional.
  - Se a BC é composta dos símbolos: $P_{1,2}$, $P_{2,2}$ e $P_{3,1}$
  - Então m_1 = {$P_{1,2}$=falsa, $P_{2,2}$=falsa, e $P_{3,1}$=verdadeira} é um modelo possível.
  - Com três símbolos proposicionais podemos ter 2^3 modelos possíveis.

Lógica Proposicional – Semântica

- Deve-se especificar como calcular o valor verdade de qualquer sentença, dado um modelo.

- Sentenças atômicas.
  - Verdadeiro é verdadeiro e Falso é falso em todo modelo.
  - O valor verdade dos outros símbolos proposicionais são especificados no modelo.
    - Ex.: $P_{1,2}$ é falsa em m_1.

- Sentenças complexas são formadas de sentenças atômicas + conectivos lógicos.

Lógica Proposicional – Semântica

- Sentenças Complexas
  - As regras para cada conectivo pode ser resumida em uma tabela-verdade.

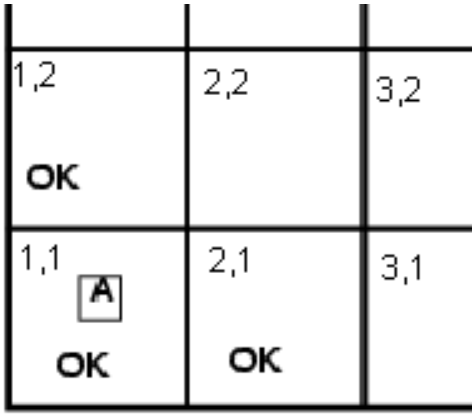
P	Q	¬P	P ∧ Q	P ∨ Q	P => Q	P <=> Q
F	F	V	F	F	V	V
F	V	V	F	V	V	F
V	F	F	F	V	F	F
V	V	F	V	V	V	V

  - $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,2})$ → $V \wedge (F \vee V) = V$
  - Uma BC é um conjunto de sentenças verdadeiras e pode ser vista como uma conjunção de todas essas as sentenças.

O mundo do *Wumpus* em Lógica Proposicional

- As regras são mais bem escritas usando-se <=>
  - Um quadrado tem brisa se e somente se um quadrado vizinho tem poço.
  - $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$

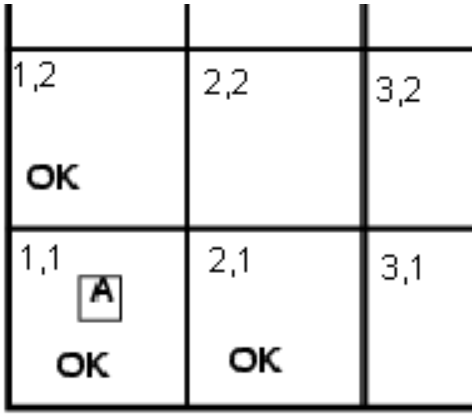
- A implicação simples $B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})$ é verdadeira, mas incompleta.
  - Ela não elimina modelos em que $B_{1,1}$ é falsa e $P_{1,2}$ é verdadeira.



Uma BC para o mundo do *Wumpus* - representando somente poços

- Vocabulário de símbolos proposicionais
  - Seja $P_{i,j}$ verdadeira se existe um poço em [i,j].
  - Seja $B_{i,j}$ verdadeira se existe brisa em [i,j].

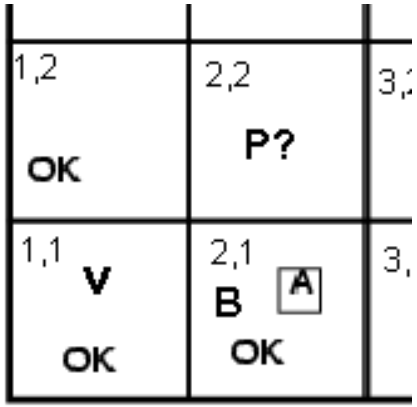
- Sentenças da Base de Conhecimento verdadeiras para todos os mundos do *Wumpus*:
  - Não há nenhum poço em [1,1]
    - S_1: $\neg P_{1,1}$
  - Um quadrado tem brisa se e somente se existe um poço em um quadrado vizinho.
    - S_2: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
    - S_3: $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
    - **Obs**.: Isto deve ser declarado para cada quadrado.



Uma BC para o mundo do Wumpus Representando somente poços

- Sentenças da Base de Conhecimento específicas para o mundo de *Wumpus* do exemplo:
  - Percepções de brisa nos dois primeiros quadrados:
    - S_4: $\neg B_{1,1}$
    - S_5: $B_{2,1}$

- A BC consiste de S_1, S_2, S_3, S_4 e S_5.
  - Ou pode ser considerada uma conjunção que afirma que todas as sentenças individuais são verdadeiras
    - $S_1 \wedge S_2 \wedge S_3 \wedge S_4 \wedge S_5$



Inferência em Lógica Proposicional

- O objetivo da inferência lógica é decidir se BC ⊨ α para alguma sentença ***α***.
- A forma mais simples de ser fazer isso é enumerar os modelos e verificar se ***α*** é verdadeira em todo modelo na qual BC é verdadeira.

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB	α_1
false	*false*	*false*	*false*	*false*	*false*	*false*	*false*	*true*
false	*false*	*false*	*false*	*false*	*false*	*true*	*false*	*true*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*false*	*true*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*false*	*true*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*false*

Equivalência Lógica

- Duas sentenças α e β são **LOGICAMENTE EQUIVALENTES** se são verdadeiras no mesmo conjunto de modelos.
  - Escrevemos isso como α <=> β .
  - **Exemplo**: p ∧ q e q ∧ p são logicamente equivalentes.
- Uma definição alternativa de equivalência é:
  - Para duas sentenças quaisquer α e β, α ≡ β, se e somente se α ⊨ β e β ⊨ α .

Equivalência Lógica

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Validade

- Uma sentença é **VÁLIDA** se ela é verdadeira em todos os modelos.
  - **Exemplo**: $(p \vee \neg p)$ é uma sentença válida
  - Sentenças válidas também são conhecidas como **tautologias** – elas são necessariamente verdadeiras e, consequentemente, vazias.
- Utilidade das sentenças válidas → derivar o teorema da dedução da definição de consequência lógica.
- **TEOREMA DA DEDUÇÃO**:
  - Para quaisquer sentenças α e β, $\alpha \vDash \beta$ se e somente se a sentença $(\alpha \Rightarrow \beta)$ é válida

Satisfazibilidade

- Uma sentença é **SATISFAZÍVEL** se é verdadeira em algum modelo.
  - **Exemplo**: a BC do exemplo é satisfazível porque existem 3 modelos onde ela é verdadeira.
  - Se a sentença **α** é verdadeira em algum modelo **m**, dizemos que **m satisfaz α** ou que **m é um modelo de α .**
- Satifazibilidade X Validade
  - **α** é válida se e somente se **¬α** é não-satisfazível.
  - **α** é satisfazível se e somente se **¬α** é não-válida.
  - **α ⊨ β** se e somente se a sentença **(α ∧ ¬β)** é não-satisfazível, este é o princípio da **PROVA POR REFUTAÇÃO**.

Padrões de raciocínio em Lógica proposicional

- Regra de inferência **Modus Ponens**

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

- Sempre que quaisquer sentenças $\alpha \Rightarrow \beta$ e α são dadas, a sentença β pode ser deduzida.

- **Exemplo**:
  - Se (***wumpusAdiante*** **Λ** ***wumpusVivo***) **=>** ***atirar*** e
  - (***wumpusAdiante*** **Λ** ***wumpusVivo***) são dadas, então podemos deduzir
  - ***atirar***

Padrões de raciocínio em Lógica proposicional

- Regra de inferência **Eliminação-de-E**

$$\frac{\alpha \wedge \beta}{\alpha}$$

- A partir de uma conjunção, qualquer elemento da conjunção pode ser deduzido.

- **Exemplo**:
  - Se ***wumpusAdiante* Λ *wumpusVivo*** é dada, então podemos deduzir
  - ***wumpusAdiante***

Padrões de raciocínio em Lógica proposicional

- Considerando os valores verdade de **α** e **β** pode-se verificar facilmente que *Modus Ponens* e Eliminação-de-E são **consistentes**.

- Estas regras podem ser usadas para gerar inferências consistentes sem a necessidade da enumeração de modelos.

- Todas as equilvalências lógicas vistas anterioremente podem ser usadas como regras de inferência.

Exemplo de aplicação das regras de inferência

- Considere a seguinte BC:
  - S_1: $\neg p_{1,1}$
  - S_2: $b_{1,1} \Leftrightarrow (p_{1,2} \vee p_{2,1})$
  - S_3: $b_{2,1} \Leftrightarrow (p_{1,1} \vee p_{2,2} \vee p_{3,1})$
  - S_4: $\neg b_{1,1}$
  - S_5: $b_{2,1}$

- Queremos **provar $\neg p_{1,2}$** a partir da BC:

- Primeiro aplicamos a eliminação do bicondicional em S_2:
  - S_6: $(b_{1,1} \Rightarrow (p_{1,2} \vee p_{2,1})) \wedge ((p_{1,2} \vee p_{2,1}) \Rightarrow b_{1,1})$

Exemplo de aplicação das regras de inferência

- S_6: $(b_{1,1} \Rightarrow (p_{1,2} \vee p_{2,1})) \wedge ((p_{1,2} \vee p_{2,1}) \Rightarrow b_{1,1})$

- Aplicamos então Eliminação-de-E em S_6:
  - S_7: $((p_{1,2} \vee p_{2,1}) \Rightarrow b_{1,1})$

- A equivalência lógica para contraposição fornece:
  - S_8: $(\neg b_{1,1} \Rightarrow \neg(p_{1,2} \vee p_{2,1}))$

- Agora podemos aplicar *Modus Ponens* com S_8 e a percepção S_4:
  - S_9: $\neg(p_{1,2} \vee p_{2,1})$

- Aplicando a regra de De Morgan, geramos a conclusão:
  - S_{10}: $\neg p_{1,2} \wedge \neg p_{2,1}$
- Aplicando Eliminação-de-E em S_{10}, geramos a conclusão:
  - S_{11}: $\neg p_{1,2}$

Provas

- Uma **PROVA** é uma demonstração mecanicamente derivável de que uma fórmula decorre logicamente de uma base de conhecimento.
- Dado um procedimento prova, **BC |– g** significa que ***g*** pode ser derivada (deduzida) da base de conhecimento BC.
- Lembre-se de que **BC |= g** significa que ***g*** é verdadeira em todos os modelos de BC.
- Um procedimento de prova é **CONSISTENTE** (*sound*) se BC |– g implica BC |= g.
- Um procedimento de prova é **COMPLETO** se BC |= g implica BC |– g.

Prova

- Descobrir provas é exatamente o mesmo que encontrar soluções para problemas de busca.
  - Função sucessora = aplicação das regras de inferência.
  - O que no pior caso não é mais eficiente do que a verificação de modelos.
  - Na prática, encontrar provas pode ser altamente eficiente porque podemos ignorar proposições irrelevantes, independente da quantidade de sentenças na BC.
    - Diferentemente da verificação de modelos.

– Propriedade de sistemas lógicos → **MONOTONICIDADE**

O que é monotonicidade?

- A **monotonicidade** afirma que o conjunto de consequências lógicas permitidas só pode ***aumentar*** à medida que as informações são acrescentadas à base de conhecimento. (Não diminuir!!!)
- Se uma base **BC |= α**
- Então ainda deve ocorrer **BC ^ β |= α**
- Ou seja, ao acrescentarmos β , o α ainda deve ser uma consequência lógica da base.

Algoritmo de prova usando regras de inferência em lógica proposicional

- Algoritmos de busca como a busca em profundidade iterativa são completas, no sentido de que eles encontrarão qualquer meta acessível.

- Mas, se as regras de inferência forem inadequadas, a meta não será acessível e o algoritmo de prova se torna incompleto.
  - **Exemplo**: Se removermos a regra de eliminação do bicondicional, a prova do exemplo anterior não seria possível.

- A regra de **RESOLUÇÃO** é uma regra de inferência única que gera um algoritmo de prova completo, quando utilizada com qualquer algoritmo de busca completo.

Regra de resolução

- Um **LITERAL** é uma sentença atômica ou a negação de uma sentença atômica, e.g. a, $\neg$b.

- Regra de **RESOLUÇÃO UNITÁRIA**

$$\alpha_1 \vee \ldots \vee \alpha_k \qquad \beta$$

$$\alpha_1 \vee \ldots \vee \alpha_{i-1} \vee \alpha_{i+1} \vee \ldots \vee \alpha_k$$

  - onde α_i e β são literais complementares (um é a negação do outro).

- Regra de **RESOLUÇÃO COMPLETA**

$$\alpha_1 \vee \ldots \vee \alpha_k \qquad \beta_1 \vee \ldots \vee \beta_n$$

$$\alpha_1 \vee \ldots \vee \alpha_{i-1} \vee \alpha_{i+1} \vee \ldots \vee \alpha_k \vee \beta_1 \vee \ldots \vee \beta_{j-1} \vee \beta_{j+1} \vee \ldots \vee \beta_n$$

  - onde α_i e β_j são literais complementares

Regra de resolução

- **Exemplo**:

$$p_{1,1} \vee p_{3,1} \qquad \neg p_{1,1} \vee p_{2,2}$$

$$p_{3,1} \vee p_{2,2}$$

- **Importante**: A cláusula resultante deve conter apenas uma cópia de cada literal.

  - **Exemplo**:
    - na resolução de (a V b) com (a V ¬b) obtemos (a V a) que será reduzido simplesmente a **a**.

Regra de resolução

- A regra de resolução é **consistente**
  - Se α_i é verdadeiro, então β_j é falso e, consequentemente $\beta_1 \vee \ldots \vee \beta_{j-1} \vee \beta_{j+1} \vee \ldots \vee \beta_n$ tem que ser verdadeira porque $\beta_1 \vee \ldots \vee \beta_n$ é dada.
  - Se α_i é falso, então $\alpha_1 \vee \ldots \vee \alpha_{i-1} \vee \alpha_{i+1} \vee \ldots \vee \alpha_k$ tem que ser verdadeira porque $\alpha_1 \vee \ldots \vee \alpha_k$ é dada.
  - Se α_i é verdadeiro ou falso, então uma dessas duas conclusões é válida, como estabelece a regra de resolução.

- A regra de resolução é **completa**
  - Qualquer algoritmo de busca completo, aplicando somente a regra de resolução, pode derivar qualquer conclusão permitida para qualquer BC em lógica proposicional.

Regra de resolução

- A resolução sempre pode ser usada para confirmar ou refutar uma sentença, mas não pode ser usada para enumerar sentenças verdadeiras.
  - **Exemplo**:
    - Dado que **a** é verdadeiro, não podemos usar a resolução para derivar **a V b.**
    - Mas podemos usar a resolução para responder à questão de **(a V b)** ser verdadeira ou falsa.
- Isso se denomina **COMPLETUDE DA REFUTAÇÃO**.

Forma Normal Conjuntiva

- Para aplicar a regra de resolução precisamos ter os literais em uma disjunção.
- Toda sentença da lógica proposicional é logicamente equivalente a uma **conjunção de disjunções de literais**.
- Uma sentença expressa como uma conjunção de disjunções de literais está na **FORMA NORMAL CONJUNTIVA** (FNC).

Procedimento de conversão para a FNC

- Vamos converter a sentença $\mathbf{b_{1,1} <=> (p_{1,2} \vee p_{2,1})}$, seguindo o procedimento:

  - Eliminar <=>, substituindo $\alpha <=> \beta$ por $(\alpha => \beta) \wedge (\beta => \alpha)$

    $(b_{1,1} => (p_{1,2} \vee p_{2,1})) \wedge ((p_{1,2} \vee p_{2,1}) => b_{1,1})$

  - Eliminar =>, substituindo $\alpha => \beta$ por $\neg\alpha \vee \beta$

    $(\neg b_{1,1} \vee p_{1,2} \vee p_{2,1}) \wedge (\neg(p_{1,2} \vee p_{2,1}) \vee b_{1,1})$

- A negação deve aparecer somente em literais, por isso temos que "movê-la para dentro" com a aplicação das seguintes equivalências lógicas:

  $\neg(\neg\alpha) \equiv \alpha$ $\quad \neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ $\quad \neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$

Procedimento de conversão para a FNC

- No exemplo, precisamos apenas da aplicação da última regra:

$$(\neg b_{1,1} \vee p_{1,2} \vee p_{2,1}) \wedge ((\neg p_{1,2} \wedge \neg p_{2,1}) \vee b_{1,1})$$

- Agora temos uma sentença que contém operadores Λ e V aninhados, aplicados a literais.
- Para transformar a sentença em uma conjunção de disjunções, vamos aplicar a distributividade de V sobre Λ sempre que possível

$$(\neg b_{1,1} \vee p_{1,2} \vee p_{2,1}) \wedge (\neg p_{1,2} \vee b_{1,1}) \wedge (\neg p_{2,1} \vee b_{1,1})$$

- Agora a sentença está na FNC, como uma conjunção de três cláusulas.
- Apesar desta sentença ser mais difícil de ler, ela pode ser usada como entrada para o procedimento de resolução.

Procedimento de conversão para a FNC

- <u>Resumo do procedimento:</u>

1) Eliminar as implicações usando formas equivalentes com $\vee$ e $\wedge$.

2) Reduzir o escopo dos sinais de $\neg$ usando a lei de De Morgan e a eliminação da dupla negação.

3) Finalizar a conversão para FNC usando as equivalências de distributividade.

Um algoritmo para a resolução

- Procedimentos de inferência baseados em resolução funcionam pelo princípio de **PROVA POR CONTRADIÇÃO**.
  - Para mostrar que **BC |= α**, mostramos que **(BC Λ ¬ α)** é não-satisfatível.

- O algoritmo funciona da seguinte forma:
  - Primeiro convertemos (BC Λ ¬ α) para FNC.
  - Depois, a regra de resolução é aplicada as cláusulas resultantes.
    - Cada par de cláusulas que tem literais complementares é resolvido para gerar uma nova cláusula, que é adicionada ao conjunto.
    - O processo continua até não existir nenhuma cláusula nova que possa ser adicionada **(BC |≠ α)**, ou derivarmos a cláusula vazia, caso em que **(BC |= α) .**

Um algoritmo para a resolução

função RESOLUÇÃO-LP(BC, α) **retorna** *verdadeiro* ou *falso*

entradas: *BC,* a base de conhecimento, uma sentença em LP

α, a consulta, uma sentença em LP

cláusulas ← o conjunto de cláusulas na representação de FNC de (BC ∧ ¬ α)

novas ← { }

Repita

// C_i, C_j são pares com literais complementares

para cada par de cláusulas C_i, C_j **em** *cláusulas* **faça**

resolventes ← RESOLVER-LP(C_i, C_j)

se *resolventes* contém a cláusula vazia: **então retornar** *verdadeiro*

novas ← *novas* U *resolventes*

se *novas* ⊆ *cláusulas*: **então retornar** *falso // não há cláusulas novas*

cláusulas ← *cláusulas* U *novas*

RESOLVER-LP(C_i, C_j) → retorna o conjunto de todas as cláusulas possíveis obtidas pela resolução de C_i e C_j.

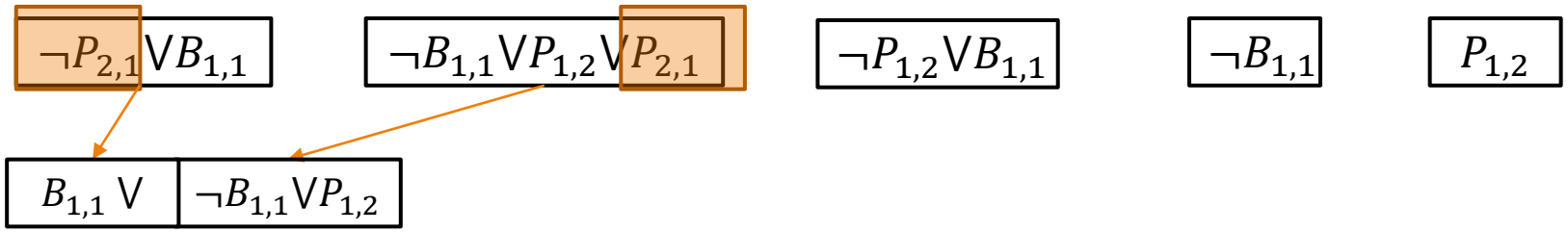
Exemplo de resolução

- Considerando a BC = $((b_{1,1} \Leftrightarrow (p_{1,2} \vee p_{2,1})) \wedge \neg b_{1,1})$.
- Desejamos provar $\neg p_{1,2}$.
- Convertendo a $(BC \wedge \neg\alpha)$ para FNC obtemos as sentenças.

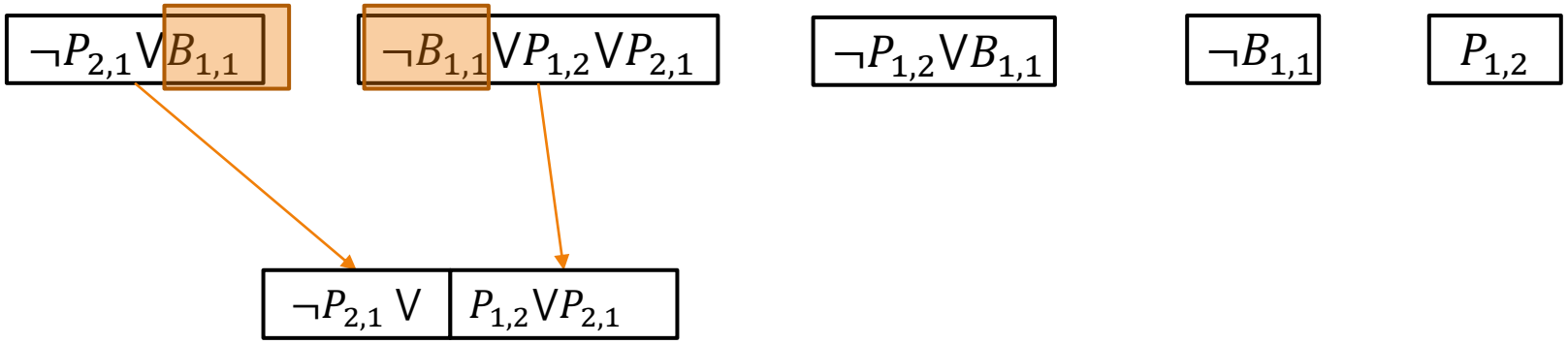
$\boxed{\neg P_{2,1} \vee B_{1,1}}$ $\boxed{\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}}$ $\boxed{\neg P_{1,2} \vee B_{1,1}}$ $\boxed{\neg B_{1,1}}$ $\boxed{P_{1,2}}$

- A seguir mostramos a resolução de pares de cláusulas.

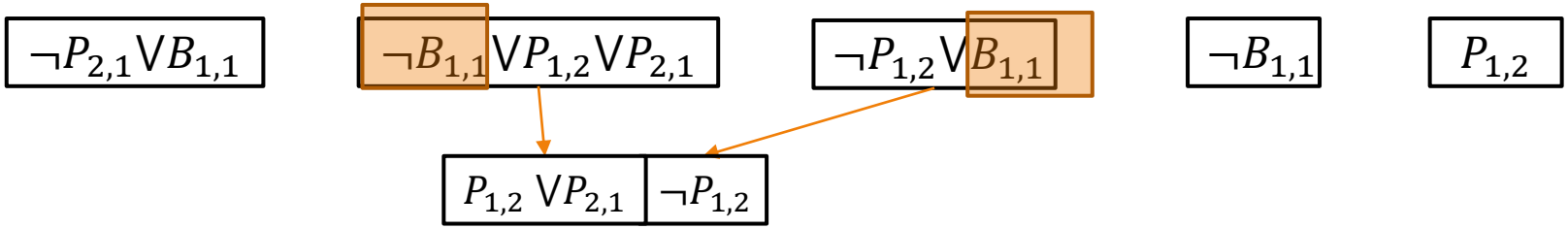
Passo 1



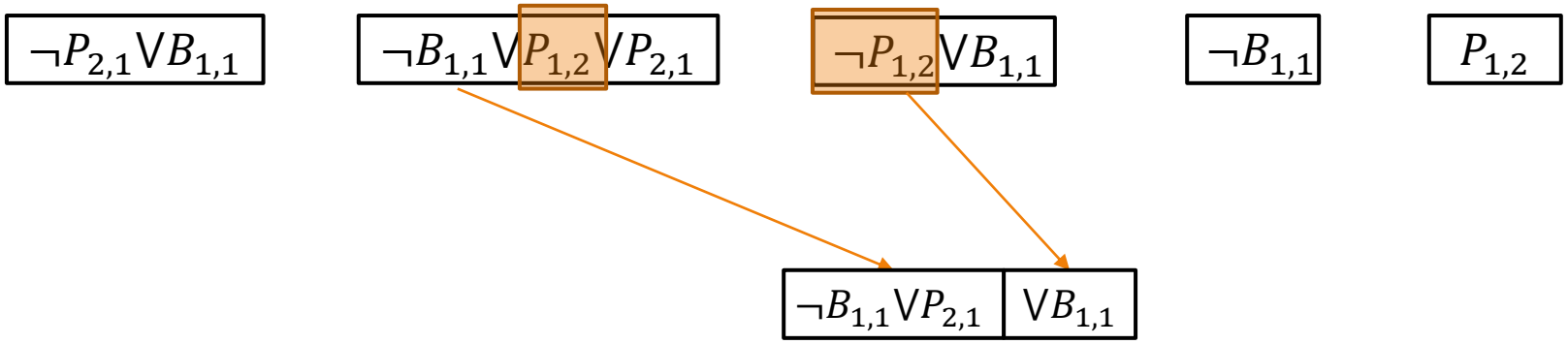
Passo 2



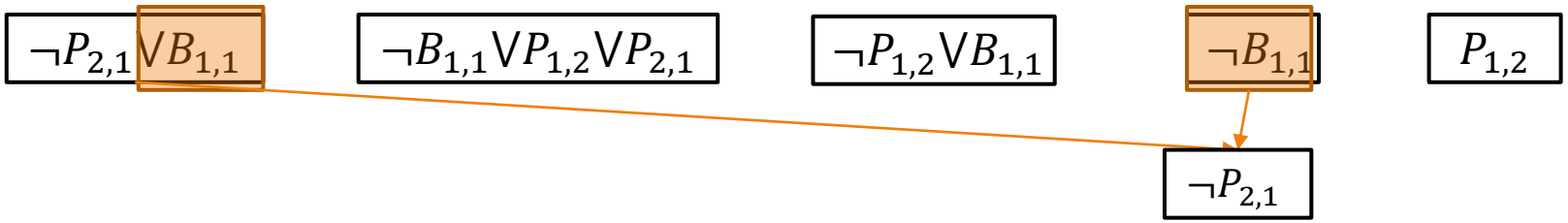
Passo 3



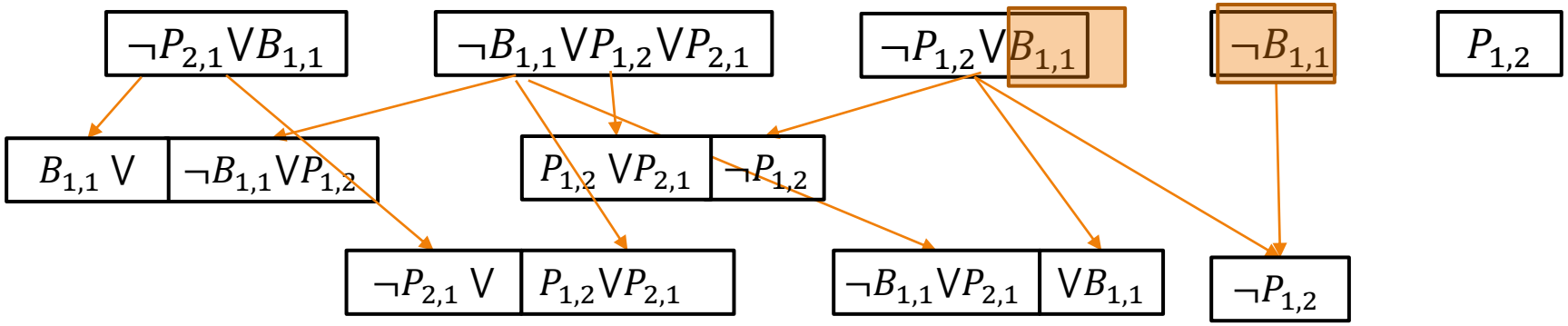
Passo 4



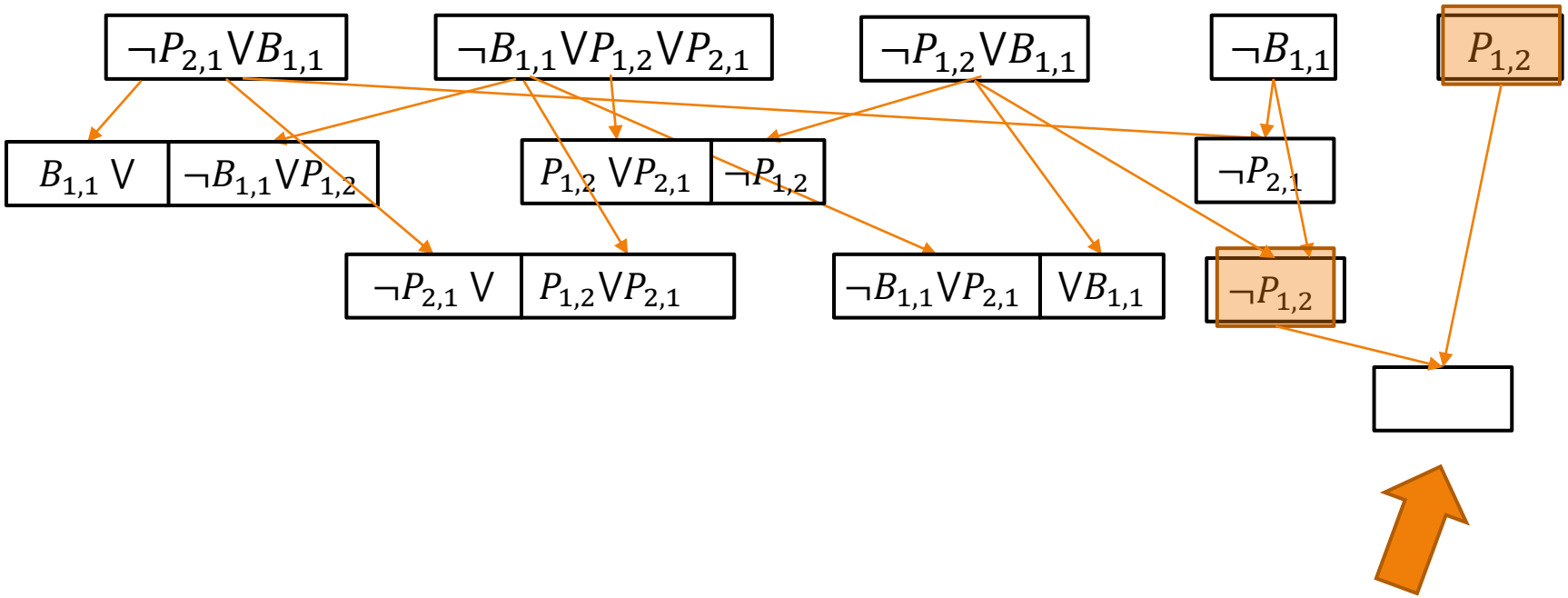
Passo 5



Passo 6



Juntando os passos de 1 a 6 mais o passo 7



PROVA POR CONTRADIÇÃO!!!

Completude da resolução

- **FECHAMENTO DA RESOLUÇÃO** FR(S) de um conjunto de cláusulas S é:
  - o conjunto de todas as cláusulas deriváveis pela aplicação repetida da regra de resolução às cláusulas em S ou suas derivadas.
  - o que o algoritmo RESOLUÇÃO-LP calcula como valor final de cláusulas.
  - **FR(S) é finito**: existe apenas um número finito de cláusulas distintas que podem ser construídas a partir do conjunto de símbolos que aparece em S.
    - Isso é verdadeiro porque existe uma etapa de fatoração que elimina várias cópias de literais de S.
    - Consequentemente RESOLUÇÃO-LP sempre termina.

Completude da resolução

- O teorema da completude da resolução em lógica proposicional é chamado de **TEOREMA BÁSICO DA RESOLUÇÃO**:
  - Se um conjunto de cláusulas é não-satisfazível, então o fechamento da resolução dessas cláusulas contém a cláusula vazia.

Bibliografia

- RUSSEL, Stuart Jonathan; NORVIG, Peter. Inteligência Artificial. Tradução por Regina Célia Simille, Artificial intelligence, 3rd ed. Rio de Janeiro: Elsevier, 2013.

Ex.:Uma implementação em Java

- url: https://github.com/aimacode/aima-java
- aima.gui.fx.applications.agent.WumpusAgentApp